Conformance Checking for Process Improvement

JM0211 Process Mining

Group 8: Rick de Rijk, Thom Verzantvoort, Stefan Vonk, Tycho van Rooij

Table of contents

1 Motivation and Scientific Background **2**

2 Conceptual Approach **2**
- 2.1 Overview . . . 2
- 2.2 Step 1: KPI definition and enrichment . . . 2
- 2.3 Step 2: Alignment and encoding . . . 3
- 2.4 Step 3: Classification . . . 3
- 2.5 Step 4: Rules and trace subsets . . . 4

3 Implementation **4**

4 Evaluation Design and Results **4**
- 4.1 Gold standards for alignments and rules . . . 4

5 Limitations and Future Work **4**

References **5**

List of Figures

1 The artefact turns a log and a model into rules in four steps. . . . 2

> **Working draft**
>
> Page limit on submission: **5 pages excluding references**. Remove this callout before the final PDF. Section ownership while drafting: Rick = Conceptual Approach (methodology) and Implementation; Tycho = literature, evaluation and results.

1 Motivation and Scientific Background

Conformance checking compares behaviour recorded in an event log with behaviour permitted by a process model in order to identify commonalities and deviations (Aalst 2016). Alignment-based conformance checking can localise deviations as *log moves*, representing observed behaviour that cannot be reproduced by the model, and *model moves*, representing modelled behaviour that was not observed.

Deviations from a normative process model are not necessarily associated with poor process performance. Dees et al. (Dees, Leoni, and Mannhardt 2017) propose a process improvement methodology in which deviations identified through conformance checking are related to a case-level KPI using classification or regression trees. Deviation patterns associated with satisfactory KPI values can subsequently inform process model enhancement, subject to business constraints.

This assignment implements the deviation analysis component of this methodology rather than the complete model repair procedure. For each case, optimal alignments against a Petri net are used to derive deviation features, which are related to case throughput time. Throughput time is defined as the elapsed time between the first and last event of a trace, following the KPI definition used in the UWV case study of Dees et al. (Dees, Leoni, and Mannhardt 2017).

Within JM0211, neighbouring topics also treat conformance (for example resource or temporal diagnostics) or process performance via predictive and prescriptive monitoring; this topic differs by linking alignment-based deviation features to a case-level KPI specifically for process improvement.

2 Conceptual Approach

2.1 Overview

The artefact implements the deviation analysis step of Dees et al. (Dees, Leoni, and Mannhardt 2017). It takes an event log in XES format and a Petri net in PNML format as input. The log must provide a case notion, an activity label for every event and a reliable timestamp for every event, because both the throughput time and the alignments depend on them. Resources and further attributes are not used. The Petri net is the normative model against which the log is compared. The artefact returns a set of classification rules that relate deviations to throughput time, together with the traces of the log that satisfy each rule.

Figure 1 shows the four steps, which the following subsections describe in order: the KPI, the alignment and encoding, the classification tree and the rule extraction. The parameters are the throughput cutoff (required), the hold-out fraction, the random seed and an optional sample size for development. The correctness of the implementation is tested against hand-built gold standards, which Section 4 describes.

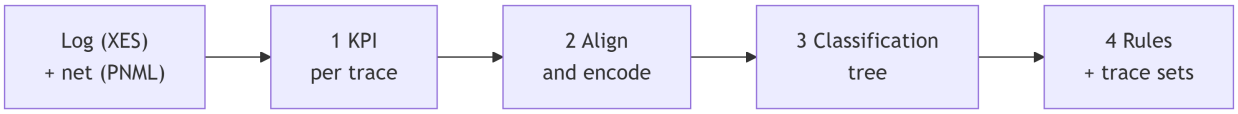


Figure 1: The artefact turns a log and a model into rules in four steps.

2.2 Step 1: KPI definition and enrichment

The first step attaches a key performance indicator (KPI) to every trace, so that deviations can later be related to how long a case takes. The brief prescribes throughput time. For a trace σ, it is the elapsed time between the first and the last event (Dees, Leoni, and Mannhardt 2017):

$$\kappa(\sigma) = t(\text{last event of } \sigma) - t(\text{first event of } \sigma).$$

The artefact stores $\kappa(\sigma)$ in days as the case attribute `throughput_time`, and lower values are better. The implementation takes the earliest and the latest timestamp of the trace, which equals the definition for

chronologically ordered traces. An empty trace or an event without timestamp raises an error, and a trace with a single event receives a throughput time of zero.

2.3 Step 2: Alignment and encoding

The second step measures how each trace deviates from the Petri net. An alignment maps a trace onto a run of the net, and every position of the alignment is a synchronous move, a log move or a model move. A synchronous move is an activity that the trace and the model perform together. A log move is an activity that the trace contains and the model does not allow at that point. A model move is an activity that the model requires and the trace lacks (Aalst 2016). Table 1 shows the three move types for a short trace.

Table 1: Log moves and model moves arise where a trace and the model disagree. The table shows one alignment of the trace A, A, C against the model A, B, C, in which the symbol ≫ marks a skipped position.

Position	1	2	3	4
Trace	A	A	≫	C
Model	A	≫	B	C
Move	synchronous	log move A	model move B	synchronous

An alignment is optimal when it has the minimal number of deviations on visible transitions, and moves on silent transitions do not count (Dees, Leoni, and Mannhardt 2017). The alignments are computed with PM4Py (Berti, Zelst, and Aalst 2019). In the version used (2.7.23.8), the default cost function and the state-equation A* solver realise this criterion: synchronous moves cost 0, log moves and model moves on visible activities cost 10000 each, and moves on silent transitions cost 1, which serves only as a tie-breaker. A trace can have several optimal alignments, and the solver returns one of them. Dees et al. argue that this choice does not affect the applicability of the methodology when the log contains many traces (Dees, Leoni, and Mannhardt 2017). Section 5 returns to this assumption.

Each alignment is then encoded as a vector of counts. For each of the 54 visible activities of the net, one feature records the number of model moves and one the number of log moves of that activity, as the brief specifies. This gives 108 features per trace. Dees et al. exclude the activities that analysts declare disallowed or mandatory from these features (Dees, Leoni, and Mannhardt 2017), but the brief provides no such constraints, so all activities are used. An activity is an event name including its lifecycle suffix, for example `W-Call-after-offers-suspend`. Synchronous moves are not features, because they carry no deviation. Silent transitions are not features either, because they never appear in the log. The counts replace the activity sequence as representation, since the 3,093 traces contain 2,038 distinct sequences and the sequence itself is therefore an unsuitable feature.

2.4 Step 3: Classification

The third step relates the encoded deviations to the KPI with a classification tree. Dees et al. use a classification tree for discrete KPI values and a regression tree for numeric ones (Dees, Leoni, and Mannhardt 2017). The brief asks for a classification tree, so the throughput time is discretised: a trace is slow if its throughput time exceeds a cutoff of 14 days, and fast otherwise. The cutoff plays the role of the set of satisfactory KPI values in Dees et al., who accept every duration below 8 days in their UWV case study (Dees, Leoni, and Mannhardt 2017). It lies close to the median of 14.43 days and yields 1,501 fast and 1,592 slow traces. It was fixed before any tree was fitted, so it does not depend on model scores. The discretisation discards the magnitude of the throughput time, which a regression tree would retain.

The tree is fitted on a stratified random sample of 75 percent of the traces, and the remaining 25 percent is held out for evaluation. A grid search with five-fold cross-validation on the training part selects the maximum depth (2 to 5), the minimum leaf size (5, 10 or 20) and the cost-complexity pruning parameter (0, 0.001, 0.01 or 0.1). This gives 48 combinations, and the grid was fixed before any scores were inspected.

The selection criterion is the F1 score of the slow class. Pruning limits the number of rules. Unpruned trees produced at least twenty rules, which is too many to interpret.

2.5 Step 4: Rules and trace subsets

The fourth step reads the fitted tree as rules. Every path from the root to a leaf is a conjunction of threshold conditions on move counts, and each leaf is one rule. The counts are integers, so a condition such as `log_move_X <= 0.5` means that activity X never occurs as a log move, and `log_move_X > 0.5` means that it occurs at least once. A rule predicts the class that the tree assigns to its leaf. For each rule, the artefact lists the identifiers of all traces that satisfy its conditions, including those whose actual class differs from the predicted one. Dees et al. keep only correctly classified traces in a cluster (Dees, Leoni, and Mannhardt 2017). The rules and trace lists cover all 3,093 traces, whereas precision and recall are computed on the hold-out set only.

3 Implementation

The artefact is implemented in Python 3.12 and organised in three packages that mirror the pipeline: data loading with KPI enrichment, alignment with encoding, and classification with rule extraction. PM4Py (Berti, Zelst, and Aalst 2019) (version 2.7.23.8) loads the XES log and the PNML net and computes the alignments. The package pandas holds the encoding table, and scikit-learn fits the tree and performs the grid search. A single entry point, `src/main.py`, runs all four steps. It requires the cutoff in days and accepts the hold-out fraction, the random seed, an optional sample size and the output paths as optional parameters. The random seed defaults to 0, so that a run is reproducible. The program writes the encoding table, the rule table and the rule-to-trace assignments as CSV files, and it can export the log enriched with throughput time as XES. All paths are relative to the project root, and `requirements.txt` pins the dependencies. Scripts in the `scripts` directory generate the two toy logs and nets and compare their alignments and rules with the hand-written gold standards.

4 Evaluation Design and Results

Owner: Tycho or Thom (lead once assigned), with group input.

The brief requires a manually built gold standard and at least precision and recall for (1) optimal alignments per trace and (2) the recovered classification rules. Dataset: BPI Challenge 2017 (Denied) as provided with the assignment; a sample may be used during development, with final numbers on the full log.

4.1 Gold standards for alignments and rules

The correctness of the alignments and the rules is tested against gold standards built by hand, because the traces of the real log are too long to align manually. Two small logs with their nets serve this purpose: a linear net with five traces, and a net with six traces that adds a parallel block and an exclusive choice. The correct move counts of every trace were derived by hand before the artefact was run. For the rules, the throughput times of selected traces were planted so that one deviation separates slow from fast traces, and the recovered trace sets are compared with the planted ones. Both toys reproduce the hand-derived move counts exactly (5 of 5 and 6 of 6 traces), and both planted rules are recovered with precision and recall of 1.0.

Results and figures on the real log: pending (Tycho).

5 Limitations and Future Work

The approach has several design limitations. Discretising throughput time at one cutoff discards the magnitude of the KPI, whereas Dees et al. use a regression tree for the numeric KPI of their UWV case (Dees,

Leoni, and Mannhardt 2017). Throughput time is elapsed wall-clock time, so it includes non-working periods and does not separate waiting from processing. An optimal alignment is also not unique, and the features depend on the one that the solver returns. Dees et al. argue that this choice does not matter when the log contains many traces (Dees, Leoni, and Mannhardt 2017), but this was not tested here.

Two further limitations concern the encoding and the rule output. Activities are lifecycle events, such as the suspension or the abortion of a work item, so many features describe interruptions and are hard to read as process deviations. The trace subset of a rule contains every trace that satisfies its conditions, including misclassified ones, whereas Dees et al. keep only correctly classified traces (Dees, Leoni, and Mannhardt 2017).

Future work follows from these limitations. A regression tree on the continuous KPI would retain its magnitude. Merging the lifecycle variants of an activity would reduce the number of features. Sampling several optimal alignments per trace would test the sensitivity to the solver's choice. Beyond the deviation analysis, Dees et al. use the selected rules to repair the log and the model, and they name the improvement of several KPIs at once and concept drift as further directions (Dees, Leoni, and Mannhardt 2017).

References

Aalst, Wil M. P. van der. 2016. *Process Mining: Data Science in Action*. 2nd ed. Springer. https://doi.org/10.1007/978-3-662-49851-4.

Berti, Alessandro, Sebastiaan J. van Zelst, and Wil M. P. van der Aalst. 2019. "Process Mining for Python (PM4Py): Bridging the Gap Between Process- and Data Science." In *Proceedings of the ICPM Demo Track 2019*, 2374:13–16. CEUR Workshop Proceedings. https://arxiv.org/abs/1905.06169.

Dees, Marcus, Massimiliano de Leoni, and Felix Mannhardt. 2017. "Enhancing Process Models to Improve Business Performance: A Methodology and Case Studies." In *On the Move to Meaningful Internet Systems. OTM 2017 Conferences*, 10573:232–51. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-319-69462-7_15.